



IFERSAN: EVENT-DRIVEN CORE

Sistema de procesamiento de ventas en tiempo real para distribuidora de bebidas en Juliaca

16,794

transacciones
procesadas

S/ 406,150

ingresos registrados

< 8 min

del ERP a Grafana



01

Registro y Extracción
ERP CasaMarket

02

Orquestación en Tiempo Real
Kafka KRaft

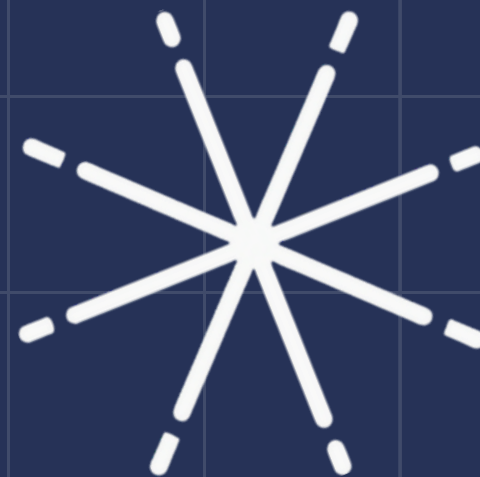
03

Procesamiento y Almacén
Spark Streaming + DB

04

ML y Observabilidad
Grafana + Prometheus

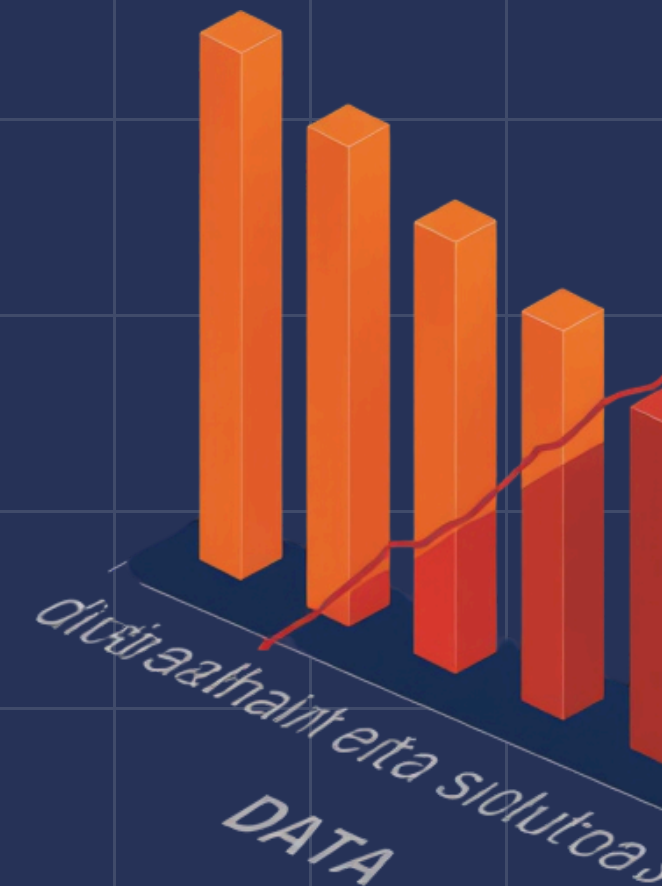




EL PROBLEMA: ¿POR QUÉ 24 HORAS ES DEMASIADO TARDE?

16,794 transacciones registradas — todas con retraso crítico

La gerencia de IFERSAN recibía datos de ventas con 24 horas de retraso vía Excel. Sin visibilidad en tiempo real, era imposible tomar decisiones comerciales oportunas, controlar el inventario o detectar anomalías de ventas en el momento crítico.



CONTEXTO Y EQUIPO

IFERSAN: Distribuidora de bebidas en Juliaca 

Alessandro Pastor — Arquitectura y pipeline completo

Cabana Sulca Cristian — Consumer, parser y Kafka

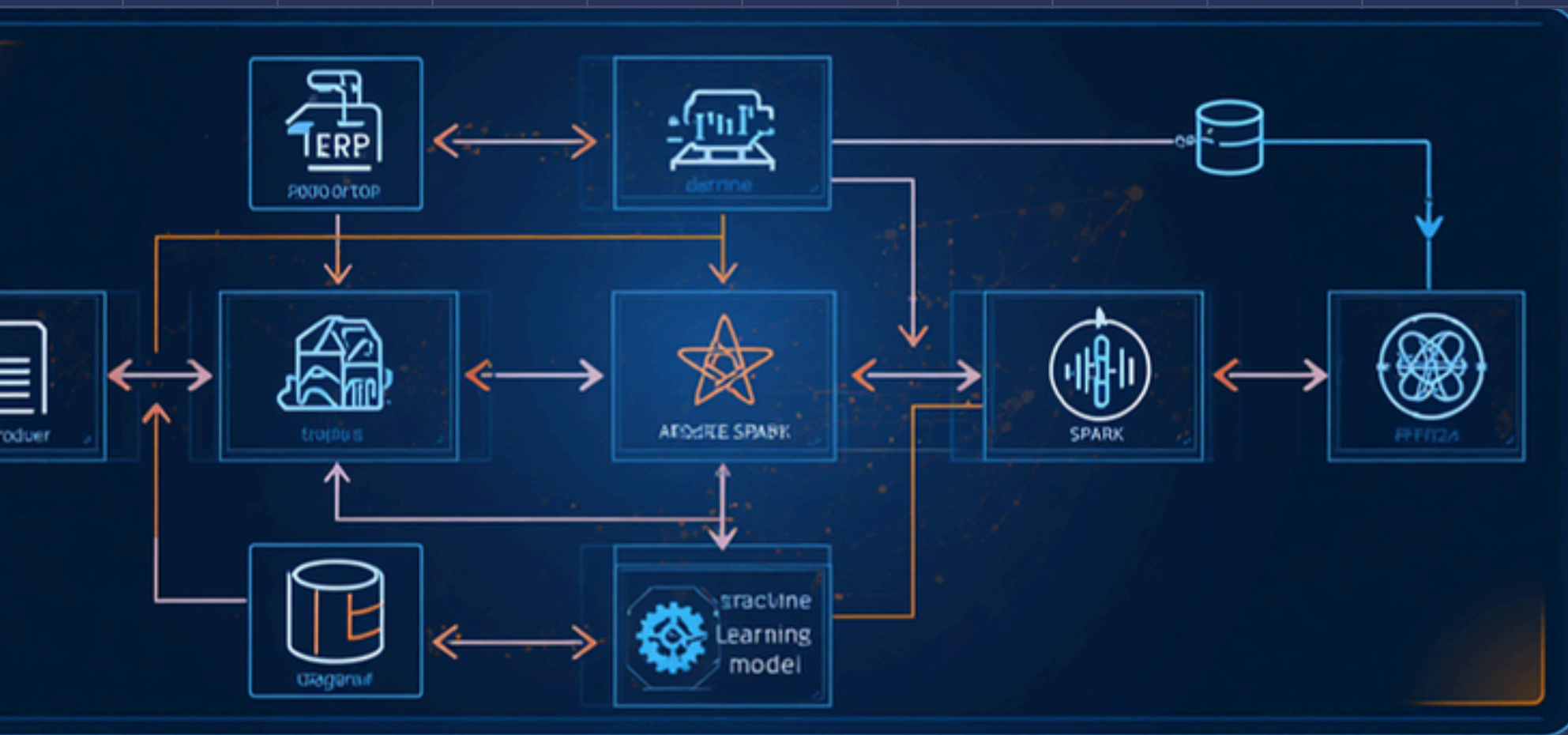
Montes Mamani Andres — Spark Streaming y PostgreSQL

 Fernandez Sanchez Jean Piero — ML, Grafana y observabilidad

Universidad Peruana Unión · IX Ciclo · Unidad 2



ARQUITECTURA KAPPA PIPELINE VISIÓN GENERAL



D. LASRMORTEREM

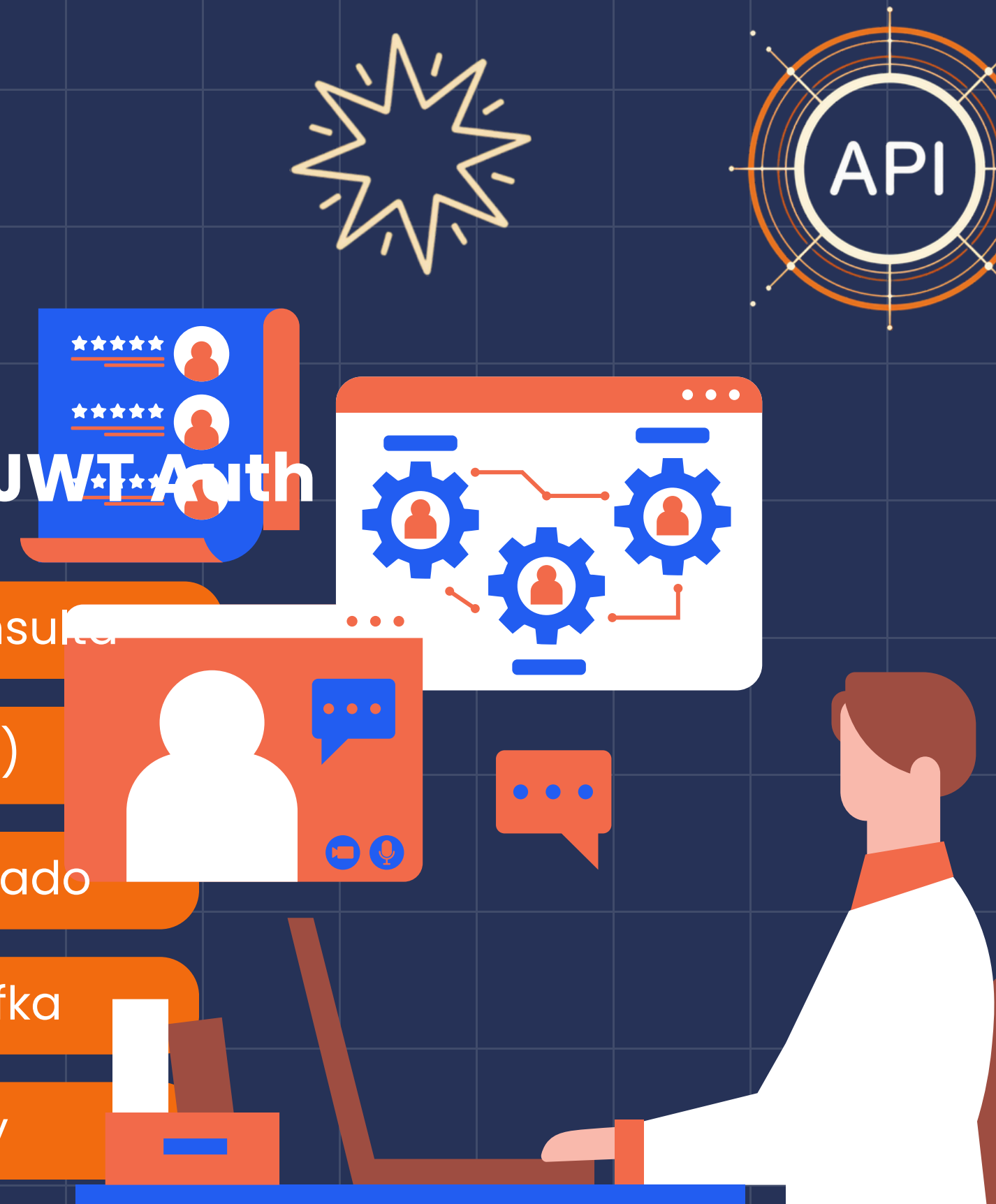
Flujo Completo de Datos en Tiempo Real

ERP CasaMarket → Producer.py → Kafka Topics → Consumidores → Spark Streaming → PostgreSQL → ML → Grafana Dashboards. Arquitectura orientada a eventos con procesamiento continuo de 16,794 transacciones.

PRODUCTOR Y FUENTE DE DATOS

Producer.py • API CasaMarket • Kafka Topic • JWT Auth

- Autenticación → JWT token renovable en cada ciclo de consulta
- Consulta API → CasaMarket ERP cada 300 segundos (5 min)
- Publicación → JSON a topic casamarket.documento.detectado
- Volumen → 175 documentos detectados y publicados a Kafka
- Ciclo continuo → Loop infinito con manejo de errores y retry



APACHE KAFKA EN MODO KRAFT

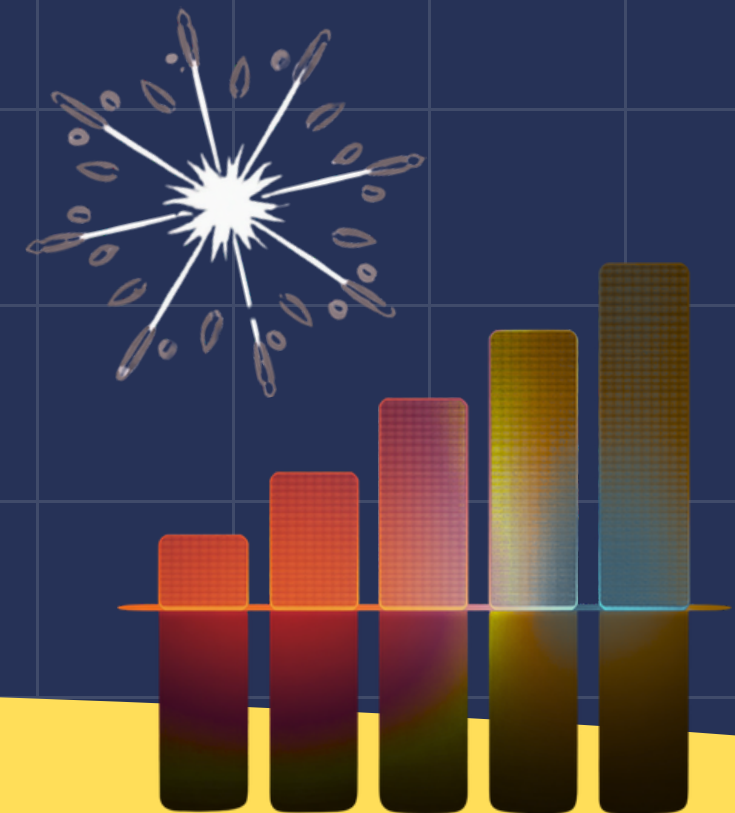


Broker sin ZooKeeper · Kafka 3.7.0 · Alta Disponibilidad

Topics activos: casamarket.documento.detectado
y casamarket.ventas.raw ·

Total: 30,372 mensajes procesados

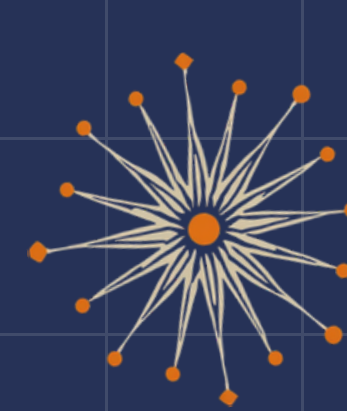
Retención: 7 días · 1 partición por topic · Modo KRaft
elimina dependencia de ZooKeeper para mayor
resiliencia.



CONSUMIDORES Y PARSING

Descarga, normalización y publicación de datos de ventas

- `consumer_downloader.py` → Descarga archivos Excel/HTML desde URLs Kafka
- `consumer_excel_parser.py` → Parsea archivos fila a fila, extrae registros
- Publicación JSON → Mensajes normalizados al topic `ventas.raw`
- Idempotencia → Archivos ya descargados y parseados no se reprocesen
- Flujo total → Kafka topic → Descarga → Parseo → `ventas.raw` → Spark



SPARK STRUCTURED STREAMING

Procesamiento en Tiempo Real con Exactly-Once

2 jobs activos: ventas y documentos — Micro-batches cada 30s, watermark 10 min, checkpoints para exactly-once. Salida a PostgreSQL, parquet y consola top 15 productos.





BASE DE DATOS POSTGRESQL

Almacenamiento Estructurado y Predicciones ML 2026

- PostgreSQL 16 con tablas ventas y predicciones_2026
- 16,794 registros de ventas reales almacenados
- 180 predicciones ML: 15 productos × 12 meses 2026
- SQL: top productos y vendedores en tiempo real
- Integración directa con Spark Streaming y Grafana

STACK TECNOLÓGICO COMPLETO

Tecnologías y Versiones del Pipeline

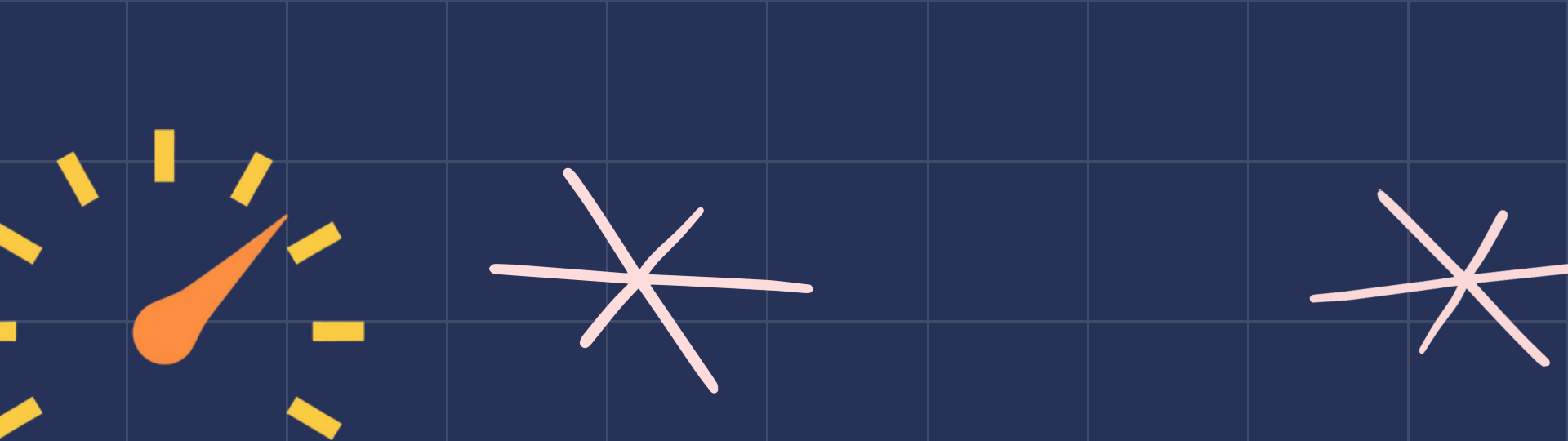
🐍 Python 3.12 · kafka-python · pandas · openpyxl

⚡ Kafka 3.7.0 KRaft · Spark 3.5.1 · PostgreSQL 16

🤖 scikit-learn (ML) · Grafana · Prometheus

🐳 Docker Compose · 13 servicios orquestados

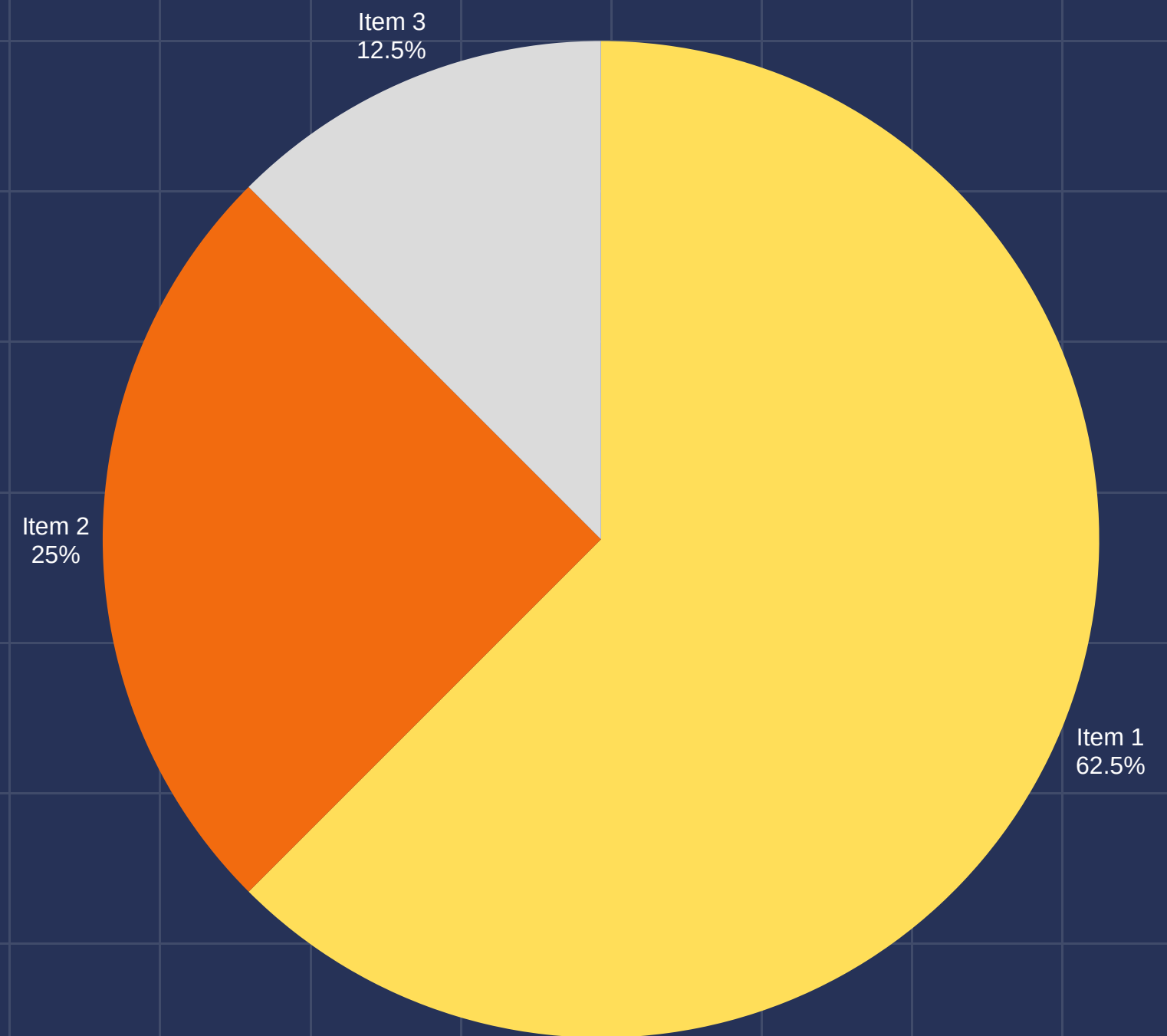




MÉTRICAS DE RENDIMIENTO

Resultados de Pruebas de Carga

Throughput: 6,074 msg/s en re-proceso ·
Consumer lag final: 0 · Trigger interval: 30s ·
Watermark: 10 min · Exactly-once garantizado
con checkpoints Spark.





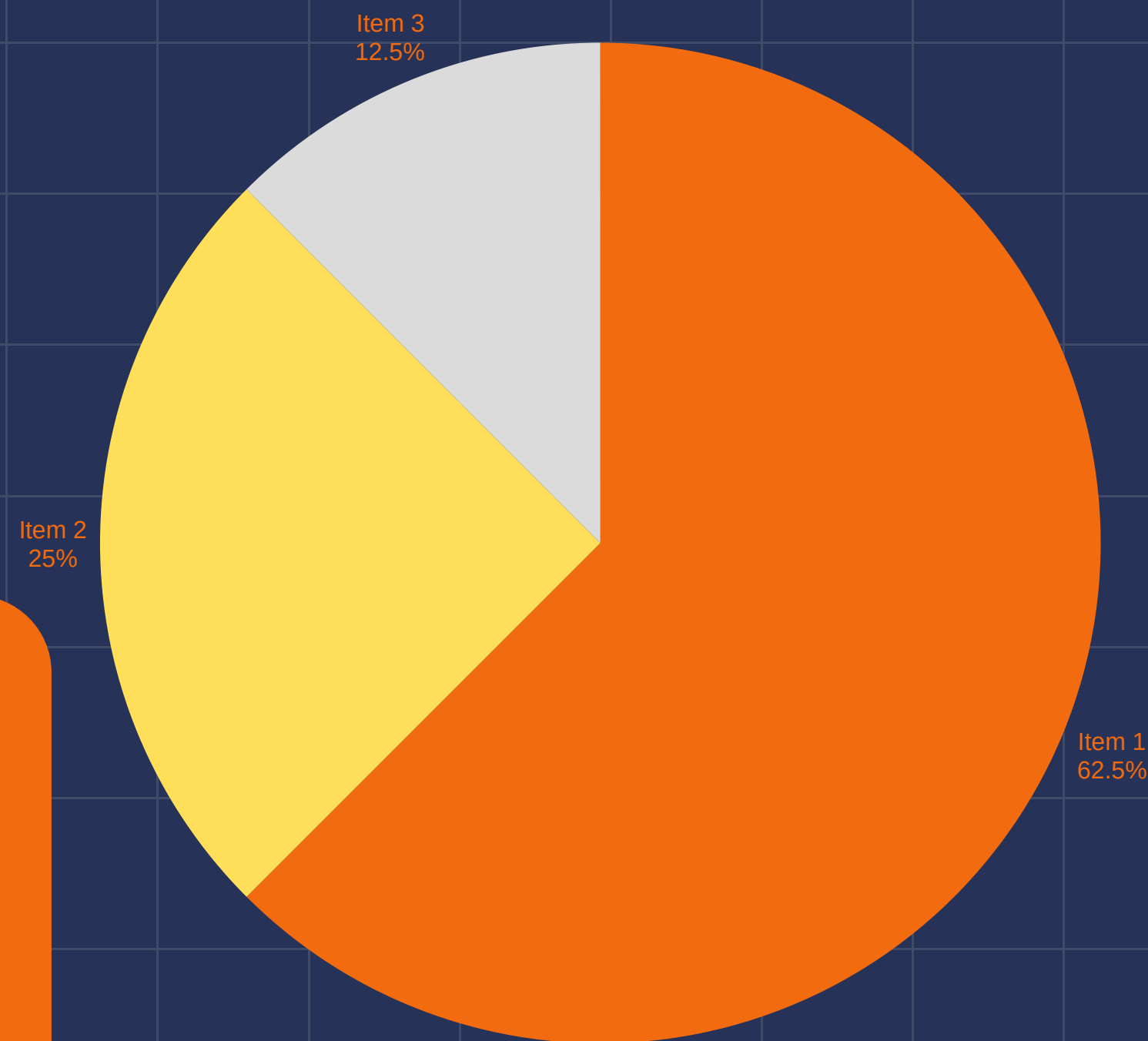
MODELO DE MACHINE LEARNING

LinearRegression · $r^2 = 0.82$ · 180
Predicciones

15 productos × 12 meses 2026 · Ventas proyectadas: S/
1,614,943.32

PEPSI 2000ML: S/ 334,800 proyectado (factor 4.4×)

scikit-learn · exactly-once pipeline · PostgreSQL
predicciones_2026



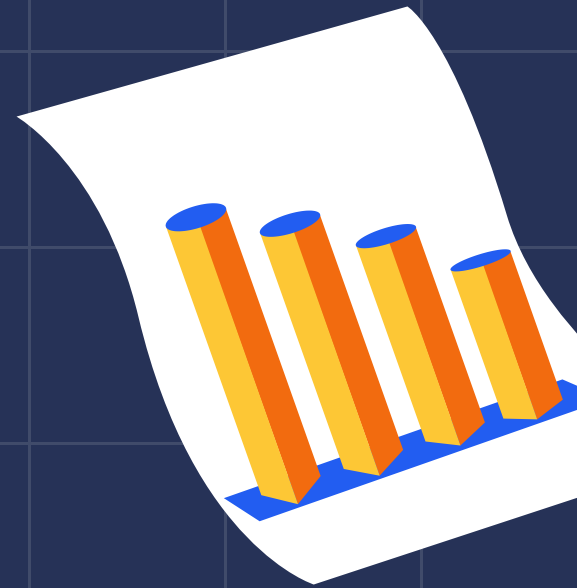
DASHBOARDS GRAFANA

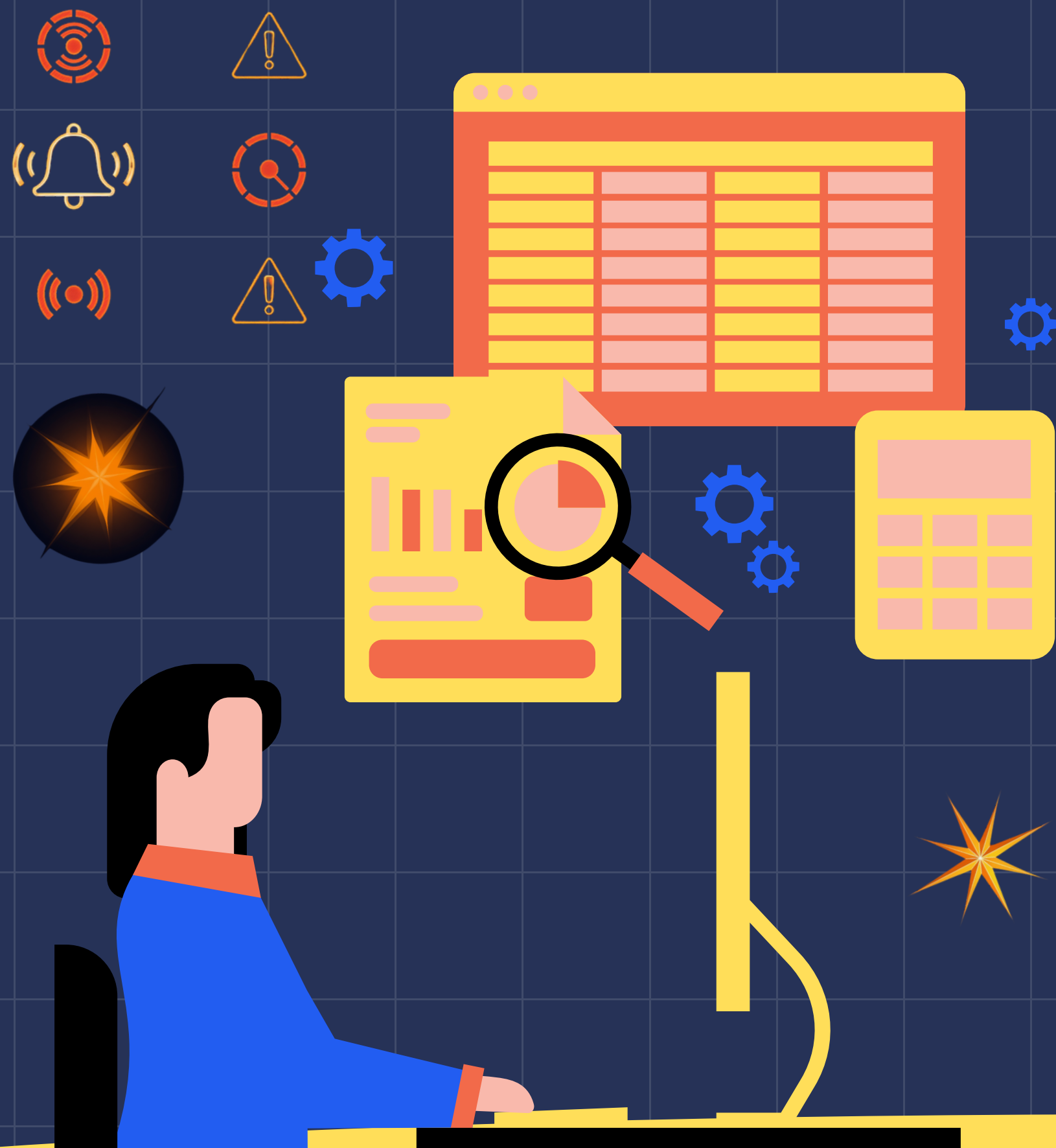
Dashboard S8 & S9: Monitoreo en Tiempo Real

S8 (9 paneles): Métricas Kafka y Spark — throughput, consumer lag, batch duration, offsets.

S9 (29 paneles): KPIs IFERSAN — Ingresos totales, transacciones, top productos, top vendedores y predicciones ML 2026.

Visibilidad completa del pipeline en menos de 8 minutos.





ALERTAS Y OBSERVABILIDAD

Monitorización Continua con Prometheus y Grafana

- KafkaConsumerLagAlto: alerta si lag > umbral crítico
- KafkaSinMensajes: detección de flujo detenido
- KafkaBrokerDown: caída del broker en tiempo real
- Notificaciones automáticas ante eventos críticos
- Observabilidad completa: métricas, logs y alertas



IMPACTO EN EL NEGOCIO

Transformación Real: De 24 Horas a 8 Minutos

- ✓ Visibilidad en tiempo real: de 24h de retraso a menos de 8 minutos
- ✓ Eliminación total de procesos manuales con Excel
- ✓ Gestión comercial en tiempo real y decisiones informadas
- ✓ Proyecciones ML para planificación 2026: S/ 1,614,943.32 proyectado

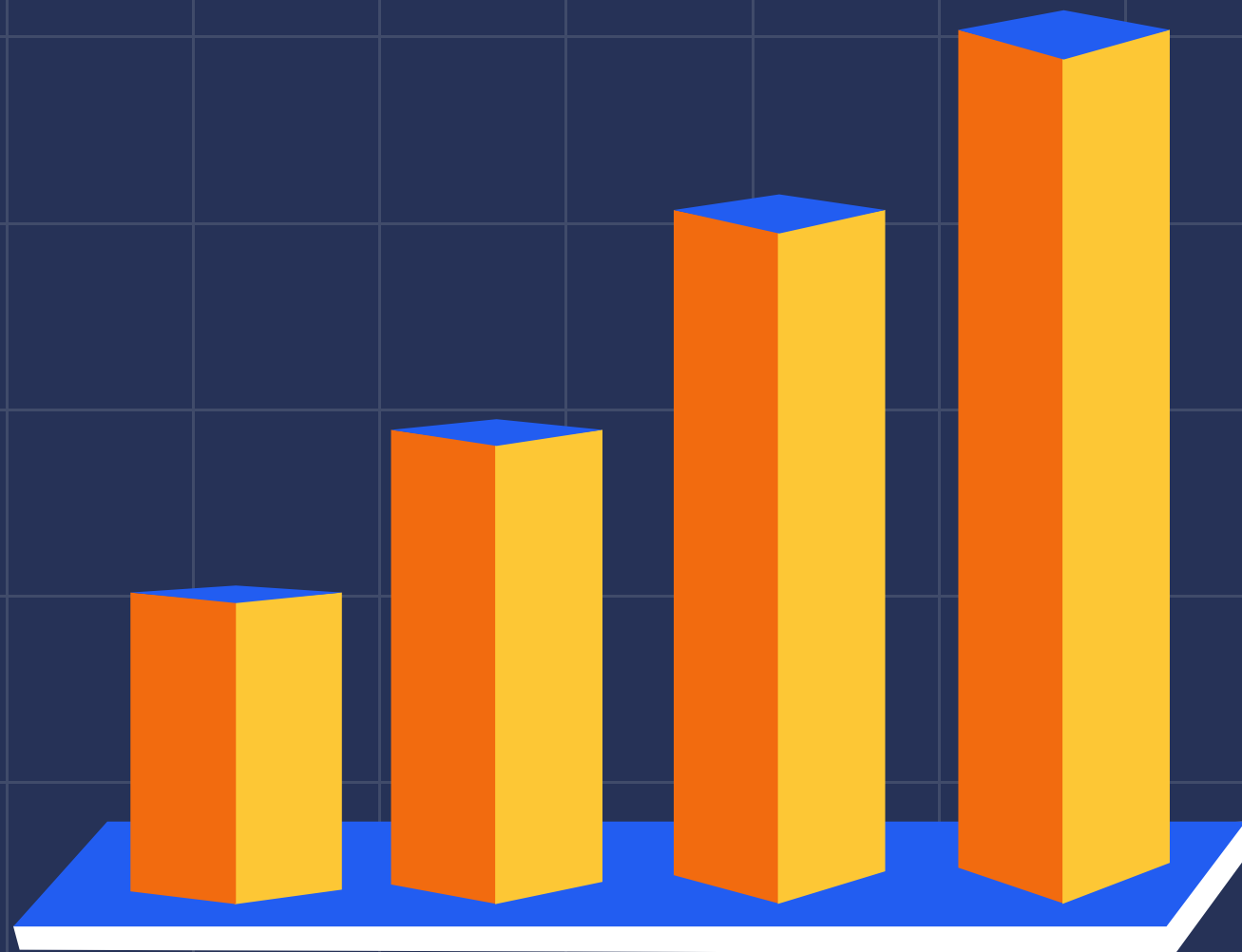


RETOS TÉCNICOS Y SOLUCIONES

Principales obstáculos encontrados y cómo fueron resueltos

- Idempotencia en Consumidores → Registro de archivos ya descargados/parseados para evitar duplicados
- Parsing Excel con Formatos Variables → Detección dinámica de columnas y manejo de filas vacías o corruptas
- Manejo BOM UTF-8 → Decodificación explícita con utf-8-sig para eliminar caracteres BOM en archivos HTML/Excel
- Checkpoints Spark (Exactly-Once) → Directorio de checkpoint persistente garantizando procesamiento sin duplicados
- Configuración KRaft sin ZooKeeper → Kafka 3.7.0 en modo KRaft con CLUSTER_ID único y metadata log propio





CÓDIGO & DEMO BREVE

Fragmentos Clave del Pipeline

producer.py: Autenticación JWT → ciclo cada 300s → publica JSON en Kafka topic casamarket.documento.detectado

prediccion_ventas.py: LinearRegression ($r^2=0.82$) → 180 predicciones × 15 productos × 12 meses 2026

Docker Compose: 13 servicios (Kafka KRaft, Spark, PostgreSQL, Grafana, Prometheus)

```
docker-compose up -d
```

```
python producer.py
```

```
python consumer_downloader.py
```

```
python consumer_excel_parser.py
```



¡GRACIAS POR SU ATENCIÓN!

Preguntas y comentarios
son bienvenidos



Universidad Peruana Unión · IX Ciclo



Big Data · Arquitectura Kappa · IFERSAN



Alessandro Pastor · Cabana Sulca Cristian



Junio 2026 · Juliaca, Perú



<https://github.com/AlessandroPastor/BigData.git>